Final Project: Wildfire Detection with CNNs

May 12th, 2026

1 Dataset Notes

I first took a look at the dataset, to see the class distribution, since that heavily affects training. It contains 7,100 images at 256×256 resolution, moderately imbalanced overall (4,893 positive, 2,207 negative). In the training split, there are 3,615 positive and 1,546 negative images. I decided to address that with inverse-frequency class weighting in the cross-entropy loss. Specifically, the weight for each class c is computed as $w_c = 1/N_c$, where N_c is the total number of training samples in class c. These raw weights are then normalized to sum to the number of classes C (i.e., $w'_c = C \cdot \frac{w_c}{\sum_j w_j}$, where $C = 2$). All images are resized to 224×224 and normalized with the ImageNet statistics from the Homework 8. Also, given the small size of the datset, I added a bit more data augmentation. I apply random resized crops (scale 0.5–1.0), horizontal and vertical flips, random rotation ($\pm 15°$), color jitter, random grayscale, and random erasing. At the batch level I use MixUp [6] ($\alpha = 0.2$) and CutMix [7] ($\alpha = 1.0$), chosen randomly each batch. These regularizers are important to make it not overfit on the dataset.

2 Simple CNN from Homework 8

I began with a six-block version of the CNN from Homework 8 (I first just gave the notebook from the homework the dataset, and the final test accuracy was under 70%, so I knew I needed a bigger model). I expanded it by adding 3 more convolution, BatchNorm, MaxPool, and ReLU blocks, and corresponding fully connected layers:

Block	**Channels**	**Kernel**	**Pool**	**Output**
Conv-BN-ReLU 1	$3 \to 16$	2×2, pad 1	MaxPool 2×2, s=1	224^2
Conv-BN-ReLU 2	$16 \to 32$	2×2, pad 1	MaxPool 2×2, s=1	224^2
Conv-BN-ReLU 3	$32 \to 64$	3×3, pad 1	MaxPool 2×2, s=2	112^2
Conv-BN-ReLU 4	$64 \to 128$	3×3, pad 1	MaxPool 2×2, s=2	56^2
Conv-BN-ReLU 5	$128 \to 256$	4×4, pad 1	MaxPool 2×2, s=2	27^2
Conv-BN-ReLU 6	$256 \to 128$	2×2, pad 1	MaxPool 2×2, s=2	14^2
AdaptiveAvgPool $\to 1 \times 1$, then FC $128 \to 32 \to 2$ with ReLU + Dropout(0.3)				

This model has exactly 755,826 parameters. Training for 50 epochs with AdamW (lr $= 5 \times 10^{-3}$, weight decay 10^{-2}), this simple CNN reached a best validation accuracy of 92.52% (epoch 40) and a test accuracy of 94.28% with test loss 0.3663.
When I was thinking of how I can improve this, there were 4 main problems that I saw the architecture didn't address, those being:

- All six convolutional blocks are purely sequential. Reading some lectures from Stanford [8] and a paper from He et al. [1], skip connections are essential for stable gradient flow in deeper networks. Without them, the gradients attenuate through six sequential nonlinearities, making it difficult to learn fine-grained features. So, in a deeper network like this, I presumably need residual connections.

- The model has to learn to detect gray smoke and flames (if present). The colors around those flames or smoke aren't very important. This CNN treats every channel equally, so being able to re-weight channels would be beneficial, perhaps.
- The model's MLP head might be too deep for just 2 classes, but also the model size in general needs to be increased.

Most of my conclusions came from giving it some other types of images not in the in dataset, like several of desert wildfires, wildfires at night, or ones with larger more expanded flames, all of which it failed on. Here are two examples:

3 More complex architecture

So, I added residual connections [1] and Squeeze-and-Excitation (SE) channel attention [2], [8]. I also changed the model so that it begins with a single 4×4 convolution at stride 4, avoiding the progressive downsampling down in the simpler model. This immediately projects the $224 \times 224 \times 3$ input into a $56 \times 56 \times 64$ feature map. The idea behind this is that rather than wasting several convolutional blocks on spatial reduction, a single strided convolution achieves the same resolution reduction in one step, and the fine-grained representative capacity likely is still there for a 224×224 image.

3.1 Residual Connections

The way I added the residual connections is via a sequence of `ResidualSEBlock`s arranged in four stages. Each block follows the pre-activation residual pattern for CNNs introduced by He et al. [1]:

$$\mathbf{y} = \sigma\big(\mathrm{SE}\big(\mathrm{BN}(\mathrm{Conv}_{3\times3}(\sigma(\mathrm{BN}(\mathrm{Conv}_{3\times3}(\mathbf{x})))))\big) + \mathbf{x}\big)$$

where σ denotes the GELU activation. I used GELU because it has no non-differentiable points and has an added bit of nonlinearity. The residual connection $\mathbf{x}$ is added to the output of the convolutional path before the final activation. When the number of channels changes between stages (e.g., $64 \to 128$), a 1×1 convolution with BatchNorm projects the shortcut to match dimensions. The residual connection hopefully makes the model more accurate by provided a direct gradient highway, so that each block only needs to learn the residual $F(\mathbf{x}) = H(\mathbf{x}) - \mathbf{x}$, which is easier to optimize than the full mapping $H(\mathbf{x})$.

3.2 Squeeze-and-Excitation Attention

Each residual block contains an SE module [2] that weights features channel-wise feature. The mechanism has two steps:

1. **Squeeze:** Global average pooling compresses each $H \times W$ feature map into a single scalar, producing a channel descriptor $\mathbf{z} \in \mathbb{R}^C$:

$$z_c = \frac{1}{H \cdot W} \sum_{i=1}^{H} \sum_{j=1}^{W} x_c(i, j)$$

2. **Excitation:** Two fully connected layers with a bottleneck learn to produce per-channel weights $\mathbf{s} \in [0, 1]^C$:

$$\mathbf{s} = \sigma_{\text{sig}}\big(W_2 \cdot \text{GELU}(W_1 \cdot \mathbf{z})\big), \qquad \tilde{\mathbf{x}}_c = s_c \cdot \mathbf{x}_c$$

This SE block, which I learned about from a Stanford lecture [8], hopefully will learn to amplify these particular channels that are the signatures of fires and smoke, while suppressing irrelevant ones (e.g., blue-sky channels, possibly water/storm clouds, etc.).

Another thing I thought about, given that this architecture has multiple residual connections and blocks is that I need to add regularization beyond just dropout for the fully-connected layers. So, I applied stochastic depth (DropPath) [3] with a linear schedule from 0 at the shallowest block to a maximum drop probability of $p_{\max} = 0.15$ at the deepest. During training, each residual branch is independently zeroed out with its assigned probability. At test time, all branches are active (scaled by their probability), recovering the full-depth model. This way, there is some regualrization for this deep architecture.

3.3 Architecture Diagram

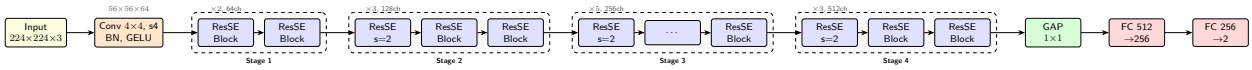

Figure 1: Architecture. Each `ResSE Block` contains two 3×3 convolutions with BatchNorm, GELU, SE attention, a residual shortcut, and stochastic depth. The first block in stages 2–4 uses stride 2 for spatial downsampling.

There are a total of 13 residual blocks and 20,006,530 parameters. The classifier is a simple two-layer head: FC $512 \to 256$ with GELU and Dropout(0.3), then FC $256 \to 2$.

3.4 Training Configuration

I trained like Homework 8 with AdamW (lr $= 10^{-3}$, weight decay $= 10^{-2}$), but with a few additions. I used cosine annealing with warm restarts (with a 3-epoch linear warmup) to help the model escape local minima. I also used gradient clipping at 1.0 to ensure training stability. Since I am using SE attention and strong augmentations like MixUp, the gradients can occasionally spike, and clipping prevents these spikes from destabilizing the weights too much. I didn't want to stop the model myself, so I added a patience parameter to my training loop to end the loop whenever there are 10 or more epochs after which the loss doesn't go down.

4 Results

The new architecture I tried above converged at epoch 37 with a best validation accuracy of **96.42%** at epoch 25. The final test evaluation yielded a loss of 0.3039 and accuracy of **97.51%**, which is a considerable improvement over the 94.28% from the simple expanded CNN. Also, this CNN aced all the extra test images that I had given the simpler CNN and it finaled. So clearly, this more complex CNN learned to generalize better over multiple type of landscapes.

4.1 Benchmarking Against Pretrained Models

I wanted to check how well this does compared to other architectures which are pretrained on ImageNet or are very versatile. I benchmarked against two architectures available through `timm`: ConvNeXt-Tiny [4] and ViT-Base/16 [5]. I tested both with transfer learning (ImageNet-pretrained) and from scratch training.

Model	Pretrained	Test Acc	Test Loss	Best Val Acc	Epochs
SimpleCNN	—	0.9428	0.3663	0.9252 (ep 40)	50
(ours)	—	**0.9751**	0.3039	0.9642 (ep 25)	37
ConvNeXt-Tiny	yes	**0.9971**	0.2600	0.9976 (ep 11)	23
ConvNeXt-Tiny	—	0.9970	0.2600	0.9975 (ep 11)	23
ViT-Base/16	yes	0.9941	0.2683	0.9952 (ep 7)	12
ViT-Base/16	—	0.6657	0.6775	0.7255 (ep 11)	23

Table 1: Summary of all model results on the wildfire detection test set.

4.2 Analysis

Clearly, ConvNeXt-Tiny does better (but not by a lot) than my model. It's interesting that it reaches the same accuracy with and without pretrained. This also shows that my model is doing pretty well. As for the Vision Transformer, with ImageNet pretraining, ViT-Base/16 reaches 99.41%, comeptitive with ConvNeXt, and as expected, because it doesn't have enough data, cannot understand image structure from our small database.

5 Future Directions

One of the main things that needs to be done, is to diversify the dataset, especially for testing. My pointed testing above needs to be done on a way larger dataset. As for model architecture, adding something like depthwise separable convolutions could be interesting. This would reduce parameter count while potentially improving performance by decoupling spatial and channel mixing. Generally trying to reduce parameter count to see how small the model can become while replicating the results of this larger, 10M model I built.

All my code is here: https://github.com/framazan/STAT-760-Final.

References

[1] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, pp. 770–778, 2016.

[2] J. Hu, L. Shen, and G. Sun, "Squeeze-and-Excitation Networks," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, pp. 7132–7141, 2018.

[3] G. Huang, Y. Sun, Z. Liu, D. Sedra, and K. Q. Weinberger, "Deep networks with stochastic depth," in *Proc. European Conf. Computer Vision (ECCV)*, 2016.

[4] Z. Liu, H. Mao, C.-Y. Wu, C. Feichtenhofer, T. Darrell, and S. Xie, "A ConvNet for the 2020s," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, pp. 11966–11976, 2022.

[5] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby, "An image is worth 16x16 words: Transformers for image recognition at scale," in *Int. Conf. Learning Representations (ICLR)*, 2021.

[6] H. Zhang, M. Cissé, Y. N. Dauphin, and D. Lopez-Paz, "mixup: Beyond empirical risk minimization," in *Int. Conf. Learning Representations (ICLR)*, 2018.

[7] S. Yun, D. Han, S. J. Oh, S. Chun, J. Choe, and Y. Yoo, "CutMix: Regularization strategy to train strong classifiers with localizable features," in *Proc. IEEE/CVF Int. Conf. Computer Vision (ICCV)*, 2019.

[8] Stanford University, "CS231n: Deep Learning for Computer Vision," Lecture 6: CNN Architectures, 2024–2025. https://cs231n.stanford.edu/2024/slides/2024/lecture_6_part_1.pdf